Certified Theorem Subsumption in Lean 4

Automating comparisons of weaker conditions, stronger conclusions, and theorem generality

Proof-of-concept design for a kernel-certified `#compare_theorems` command

September 2026 | Lean 4 metaprogramming and proof automation

Executive summary. For similarly shaped Lean theorems, "more general assumptions" and "stronger conclusions" can be reduced to ordinary proof obligations. If Lean proves those obligations, the comparison is certified by the kernel. The practical tool is therefore not a semantic oracle, but an *obligation generator plus proof-search portfolio*: inspect theorem types, align parameters, transport contexts, generate directional implication goals, run automation, retain proof terms, and report **proved**, **refuted when certified**, or **unknown**. This supports theorem-library cleanup, lemma discovery, API evolution, and redundancy detection without weakening Lean's trust model.

1 Problem statement

Suppose two theorems have the common shape

$$T_1 : \forall x,\ A_1(x) \to C_1(x), \qquad T_2 : \forall x,\ A_2(x) \to C_2(x).$$

For aligned inputs x:

- T_1 **has weaker (more general) conditions than** T_2 when

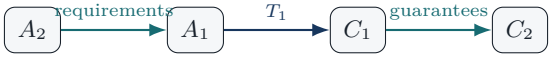
  $$\forall x,\ A_2(x) \to A_1(x).$$

  Anything admitted by T_2 is also admitted by T_1.
- T_1 **has a stronger (more specific) conclusion than** T_2 when

  $$\forall x,\ C_1(x) \to C_2(x).$$

  Anything established by T_1 entails what T_2 promises.

This is the familiar variance rule: requirements are **contravariant**; guarantees are **covariant**.



A component-wise subsumption judgement can therefore be presented as four directional checks:

Question	Proof obligation	Interpretation if proved
Requirements	$A_2 \to A_1$	T_1 requires no more than T_2
Requirements	$A_1 \to A_2$	T_2 requires no more than T_1
Conclusions	$C_1 \to C_2$	T_1 guarantees at least what T_2 does
Conclusions	$C_2 \to C_1$	T_2 guarantees at least what T_1 does

If both directions hold for one dimension, that dimension is equivalent. If exactly one direction is certified, the ordering is strict. If neither direction is proved, the correct default is *unknown or incomparable*, not "false."

2 Worked examples

2.1 Arithmetic: strict subsumption

```
theorem T1 (x : Int) (h : x > 0) : x >= 1 := by
  grind

theorem T2 (x : Int) (h : x > 10) : x >= 0 := by
  grind
```

The generated comparison goals are essentially:

```
-- T1 has weaker assumptions.
example (x : Int) : x > 10 -> x > 0 := by
```

```
  grind

-- T1 has the stronger conclusion.
example (x : Int) : x >= 1 -> x >= 0 := by
  grind
```

Both implications are provable, so T_1 component-wise subsumes T_2. The tool should retain the proof expressions it found, making the classification independently kernel-checkable.

2.2 Whole-theorem implication is useful, but not enough

A second check is direct theorem implication:

$$T_1 \to T_2.$$

This asks whether a proof of one theorem can be adapted into a proof of the other. It is an excellent end-to-end check, but it can hide interactions between premises and conclusions.

For example:

```
-- A proof of T1 can be adapted to the stronger-looking T2
-- because P is available inside T2's premise.
example (P Q : Prop) (h : P -> Q) : P -> And P Q := by
  intro hP
  exact And.intro hP (h hP)
```

At the level of propositions, $(P \to Q) \to (P \to P \wedge Q)$ is provable: inside the premise P, one already has the extra P needed in the stronger-looking conclusion. But $Q \to P \wedge Q$ is not valid without that premise. Thus whole-theorem implication and context-free conclusion comparison answer different questions.

Recommendation. Report both. Use *component-wise subsumption* for the informative "weaker requirements / stronger guarantee" judgement, and whole-theorem implication as a secondary adapter check.

2.3 Typeclass hierarchies: conditions are not only propositions

Real theorems often have telescopes such as:

```
T1 : forall {a : Type} [Preorder a]
       (f : a -> a) (hmono : Monotone f), C1 f

T2 : forall {a : Type} [LinearOrder a]
       (f : a -> a) (hstrict : StrictMono f), C2 f
```

Here a requirement is a *constructible context item*, not merely a proposition. Under `[LinearOrder a]`, typeclass synthesis can normally construct the parent `Preorder a`; similarly, proof automation may establish `Monotone f` from `StrictMono f`. The generalized relation is therefore a context transformation

$$\Gamma_2 \longrightarrow \Gamma_1$$

followed by a conclusion transformation $C_1 \to C_2$ in the transported context.

3 A more general formal model: telescope transport

Lean theorem statements are dependent function (Pi) types. There is no universal syntactic boundary labeled "parameters end here; assumptions begin here." A robust comparator should model each theorem as an ordered telescope of binders followed by a result:

$$T_i \equiv \Pi(b_1^i : B_1^i) \ldots (b_n^i : B_n^i),\ C_i.$$

Some binders are data inputs, some are proposition-valued proofs, some are instance implicits, and some are dependent witnesses.

After a shared input alignment Δ is chosen, say that T_1 has **no stronger requirements** than T_2 if, from the context of T_2, the comparator can construct all remaining inputs needed to instantiate T_1 in order. This construction is a dependent *context transport*:

$$\phi : \Gamma_2 \rightsquigarrow \Gamma_1.$$

Then T_1 has a **stronger guarantee relative to that transport** if it can prove

$$\Delta, \Gamma_2, h : C_1[\phi] \ \vdash\ C_2.$$

Together, ϕ and this proof form an explicit adapter from T_1 to T_2.

This formulation naturally handles ordinary hypotheses, typeclasses, reordered or derivable requirements, dependent binders, and stronger algebraic structures. It also exposes the hard part: **choosing the alignment of genuinely shared inputs**.

3.1 Parameter alignment must be explicit in the design

A practical comparator should support three modes:

1. **Positional MVP.** Align a common prefix of non-proof inputs by position, accepting types up to definitional equality. This covers many families of sibling lemmas.
2. **Heuristic alignment.** Match candidates by binder information, dependency shape, and definitional equality, with bounded backtracking.
3. **User override.** Allow an explicit mapping when theorem interfaces differ materially. This prevents a heuristic mismatch from being presented as a logical result.

Names should be advisory only: alpha-renaming must not affect the judgement.

4 What can and cannot be automated

4.1 Sound positive answers

Lean metaprogramming is well suited to orchestration. The metaprogram can inspect theorem types as `Expr`, open Pi binders with `forallTelescopeReducing`, compare or unify expressions using `isDefEq`, synthesize typeclass requirements with `synthInstance`, create metavariable goals, and invoke tactics programmatically. Lean's kernel then checks every generated proof term. The trusted claim is therefore not "the heuristic says yes," but "the comparator produced a proof of the comparison obligation."

4.2 No complete semantic decision procedure

For arbitrary propositions, deciding whether $P \to Q$ is provable is itself an arbitrary theorem-proving problem. Metaprogramming cannot remove this fundamental limitation. A production result type should therefore distinguish at least:

```
inductive ProofStatus where
  | proved    -- contains / references a proof certificate
  | refuted   -- only when a proof of the negation is found
  | unknown   -- search exhausted, timed out, or alignment ambiguous
```

"Automation failed" must never be silently converted into "the implication is false." A refutation is justified only when Lean proves the negation (or a sound decision procedure supplies a certificate).

4.3 Automation portfolio

For the generated obligations, a bounded portfolio can proceed from cheap to expensive:

1. definitional equality / exact local assumption;
2. typeclass synthesis for instance-implicit targets;
3. simplification and deterministic local reasoning;
4. `grind` as a strong general-purpose engine;
5. theory-specific solvers such as linear arithmetic; in Mathlib projects, optionally `aesop`, `linarith`, and domain tactics;
6. optional project-provided tactic hooks.

Modern `grind` is especially attractive because it combines SMT-inspired equality reasoning, theorem instantiation, case analysis, and specialized solvers while still producing proof terms checked by the kernel.

5 Proof-of-concept design

5.1 User surface

The minimal interface is a command:

```
#compare_theorems T1 T2
```

Illustrative output:

```
Comparing T1 against T2

Shared inputs:
  matched: x : Int

Requirements:
  T2 => T1 : PROVED   [grind]
  T1 => T2 : UNKNOWN
  result   : T1 has weaker requirements

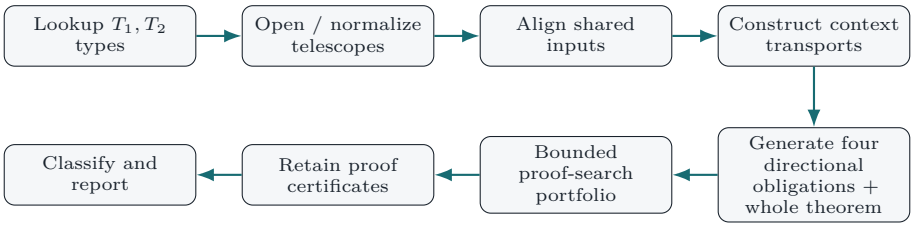
Conclusions:
  T1 => T2 : PROVED   [grind]
  T2 => T1 : UNKNOWN
  result   : T1 has stronger conclusions

Whole theorem:
  T1 => T2 : PROVED

Overall:
  T1 SUBSUMES T2 (certified in the proved directions)
```

Useful options could include a heartbeat budget, alignment strategy, an explicit binder mapping, a tactic portfolio, and a verbose mode that prints generated obligations and proof terms.

5.2 Pipeline



5.3 Core data model

```
structure BinderView where
  fvar       : Expr
  type       : Expr
  binderInfo : BinderInfo
  isProp     : Bool

structure Alignment where
  -- Mapping between genuinely shared inputs.
  leftToRight : Array (Nat x Nat)
  confidence  : AlignmentConfidence

structure Attempt where
  status     : ProofStatus
  proof?     : Option Expr
  tactic?    : Option String
  elapsedMs  : Nat

structure ComparisonReport where
  req_2_to_1   : Attempt
  req_1_to_2   : Attempt
  concl_1_to_2 : Attempt
  concl_2_to_1 : Attempt
  whole_1_to_2 : Attempt
  whole_2_to_1 : Attempt
```

The exact record layout is secondary; the important invariant is that every `proved` edge carries or can reconstruct a proof certificate.

5.4 Algorithm sketch

```
compare(T1, T2):
  ty1 <- typeOf(T1)
  ty2 <- typeOf(T2)
```

```
tel1, C1 <- forallTelescopeReducing(ty1)
tel2, C2 <- forallTelescopeReducing(ty2)

alignment <- alignSharedInputs(tel1, tel2)

-- Direction: T2 requirements can satisfy T1 requirements.
req_2_to_1 <- buildTransport(source = tel2,
                             target = tel1,
                             alignment)

-- Symmetric requirement check.
req_1_to_2 <- buildTransport(source = tel1,
                             target = tel2,
                             reverse(alignment))

-- If transport exists, instantiate C1 and C2 in a common context
-- and prove the directional conclusion implications.
concl_1_to_2 <- prove(C1[req_2_to_1] -> C2)
concl_2_to_1 <- prove(C2[req_1_to_2] -> C1)

whole_1_to_2 <- prove(typeOf(T1) -> typeOf(T2))
whole_2_to_1 <- prove(typeOf(T2) -> typeOf(T1))

return classify(all attempts)
```

5.5 How `buildTransport` should work

Given the source theorem's instantiated context, process target binders in dependency order. For each target requirement:

1. Reuse an explicitly aligned shared input if one exists.
2. Otherwise search current locals for a term whose type is definitionally equal to the required type; `isDefEq` may instantiate metavariables during alignment.
3. If the binder is instance-implicit, try `synthInstance`.
4. If its type is a proposition, create a proof goal and run the bounded tactic portfolio.
5. If it is a non-propositional dependent witness not designated as a shared input, attempt configured constructors/search; otherwise return `unknown` and explain which binder could not be synthesized.
6. Add the constructed term to the target instantiation so later dependent binders can refer to it.

This is more general than flattening all hypotheses into a conjunction and preserves dependent telescope semantics.

5.6 Lean metaprogramming hooks

The PoC can be implemented as a command elaborator that lifts into term elaboration / `MetaM`. The key facilities are:

Lean facility	Role in the comparator
`Expr`	representation of theorem types and generated obligations
`forallTelescopeReducing`	expose dependent Pi binders modulo reduction
`isDefEq`	match types / expressions up to definitional equality and unification
`isProp`	classify proposition-valued binder types
`synthInstance / synthInstance?`	construct typeclass requirements
`mkFreshExprMVar`	allocate proof goals for generated obligations
`Elab.Tactic.evalTactic`	programmatically run tactic syntax against goals
`Elab.Command.liftTermElabM`	bridge a custom command into elaboration / MetaM

6 Engineering considerations

6.1 Search bounds and reproducibility

The command should be deterministic under a fixed Lean/project version, tactic set, options, and heartbeat budget. Each attempt should record the tactic path and resource cap. Expensive backtracking in parameter alignment should be separately bounded from proof search.

6.2 Do not over-interpret binder syntax

A binder of type `Prop` is usually an assumption, but theorem APIs may use proposition-valued data deliberately; a non-`Prop` binder may be a witness requirement rather than a conceptual parameter. Therefore automatic role classification is a convenience, not a semantic truth. Explicit mappings are essential for difficult cases.

6.3 Certificates over scores

Heuristic confidence scores can help choose an alignment, but should not be the basis of the logical claim. The report should visually separate:

- **alignment decisions** (possibly heuristic), from
- **proved edges** (kernel-certified), from
- **unknown edges** (search did not settle them).

6.4 Library-scale use

For a corpus of lemmas, the pairwise relation can be viewed as a directed graph. A certified edge $T_i \to T_j$ indicates a reusable adapter. Component-wise subsumption edges are even more useful for detecting dominated lemmas. To control quadratic cost, first cluster theorems by head symbols, result type skeletons, namespaces, or normalized telescope fingerprints, then run semantic checks only inside plausible clusters.

7 MVP scope and evaluation plan

A credible first implementation can be deliberately narrow:

1. Same universe / data parameter prefix, aligned positionally up to `isDefEq`.
2. Remaining instance-implicit and proposition-valued binders treated as requirements.
3. Generate both requirement directions, both conclusion directions, and both whole-theorem directions.
4. Solver portfolio: exact/assumption, typeclass synthesis, simp, then `grind`; Mathlib adapter optionally adds `aesop`, `linarith`, etc.
5. Results are `proved`, `refuted`, or `unknown`; never infer falsehood from timeout/failure.
6. Print the exact generated obligations in verbose mode.

Evaluate on curated theorem families with known relationships: arithmetic bounds, monotonicity lemmas, order/algebra hierarchy variants, stronger/weaker side conditions, and deliberately incomparable pairs. Measure (i) certified comparisons found, (ii) false logical classifications – which should be zero because positive claims require proofs, (iii) unknown rate, (iv) alignment errors caught by diagnostics, and (v) runtime / heartbeats.

Success criterion for the PoC. It does not need to solve all theorem comparisons. It needs to reliably discover a useful subset, explain its alignment, emit the obligations it tried, and attach Lean-checkable evidence to every positive judgement.

8 Roadmap beyond the PoC

- **V1:** positional alignment, proposition/typeclass requirements, proof portfolio, rich report.
- **V2:** bounded structural matching for reordered inputs and dependent telescope transport; user-specified binder maps.
- **V3:** domain plugins, counterexample/refutation backends where soundly available, proof caching, and theorem-corpus indexing.
- **V4:** library tooling: "find lemmas subsumed by this theorem," "find a stronger theorem I can use," and CI linting for accidental duplicate APIs.

9 Conclusion

The proposed comparator is feasible precisely because it does *not* try to decide an informal notion of theorem quality directly. It translates a selected notion of generality into explicit Lean propositions and context-construction problems. Metaprogramming supplies the plumbing; tactics supply incomplete but powerful proof search; the kernel supplies trust. The central design principle is simple: **automate discovery, certify conclusions, and report uncertainty explicitly.**

References and implementation notes

1. Lean Language Reference, current documentation and release notes. The release index lists Lean 4.33.1 (2026-08-21) and 4.34.0 release candidates at the time of writing. https://lean-lang.org/doc/reference/latest/
2. Lean Language Reference, *Tactic Reference*; documentation for `grind` and its SMT-inspired reasoning / integrated solvers. https://lean-lang.org/doc/reference/latest/Tactic-Proofs/Tactic-Reference/
3. *Metaprogramming in Lean 4*, MetaM chapter; discussion of `isDefEq`, telescopes, and `forallTelescopeReducing`. https://leanprover-community.github.io/lean4-metaprogramming-book/main/04_metam.html
4. Lean API documentation, `Lean.Meta.SynthInstance`; `synthInstance`, `synthInstance?`, and related typeclass search facilities. https://lean-lang.org/doc/api/Lean/Meta/SynthInstance.html
5. Lean API documentation, `Lean.Elab.Tactic.Basic`; programmatic tactic evaluation via `evalTactic`. https://lean-lang.org/doc/api/Lean/Elab/Tactic/Basic.html
6. Lean API documentation, `Lean.Elab.Command`; `liftTermElabM` for running elaboration / MetaM work from a command elaborator. https://lean-lang.org/doc/api/Lean/Elab/Command.html

Version note. API names cited here are current as of 1 September 2026. The pseudocode is an architectural PoC sketch, not a claim that the listing compiles unchanged against every Lean release or Mathlib pin. A real package should pin Lean and test the command against that version.